



Технически Университет – София
Филиал Пловдив

**Семинарно упражнение по дисциплина
„Писане на сигурен код с Python“**

Семинарно упражнение №4

Тема: Реализация и анализ на Path Traversal
уязвимост в Django приложение

I. Цел на упражнението

Целта на упражнението е запознаване с уязвимостта Path Traversal, демонстриране на нейното проявление при неконтролирана работа с файлови пътища и прилагане на подход за сигурно ограничаване на достъпа само до позволена директория.

II. Стъпки за реализация на упражнението

Стъпка 1) Създайте Django проект и приложение

Създайте нов Django проект и приложение, в което ще бъде реализирана функционалност за четене на файлове. Това приложение ще съдържа както уязвима, така и защитена версия на достъпа до файлове, за да може да се сравни поведението им при нормално използване и при опит за атака.

Стъпка 2) Създайте директория за файловете и примерни текстови файлове

Добавете директория, от която приложението ще чете файлове. Тя ще играе ролята на разрешена зона за достъп. В нея създайте няколко примерни текстови файла, които ще се използват за нормално тестване на функционалността през брауъра.

Стъпка 3) Създайте HTML шаблоните на приложението

Добавете HTML шаблони, чрез които ще се достъпват началната страница, уязвимата версия и защитената версия на четене на файлове. Така упражнението ще може да се демонстрира по-лесно през брауър, а не само чрез директно извикване на URL адреси с параметри.

Стъпка 4) Реализирайте страниците за визуализация на шаблоните

Добавете изгледи, които ще визуализират началната страница, страницата на уязвимата версия и страницата на защитената версия. Тези изгледи не извършват файлово четене, а само връщат съответния HTML шаблон, през който по-късно ще се подава името на файла.

```
def home(request):
    return render(request, "file_app/home.html")

def vulnerable_page(request):
    return render(request, "file_app/read_file_vulnerable.html")

def safe_page(request):
    return render(request, "file_app/read_file_safe.html")
```

Стъпка 5) Реализирайте уязвимата версия за четене на файл

Добавете изглед, който приема името на файла чрез GET параметър и директно го използва за отваряне на файл в директорията uploads. Тази реализация е уязвима, защото не прави проверка дали подаденият път остава в разрешената директория и по този начин позволява излизане извън нея.

```
def read_file_vulnerable(request):
    filename = request.GET.get("file")

    if not filename:
        return HttpResponse("No file specified.")

    with open(f"uploads/{filename}", "r", encoding="utf-8") as f:
        content = f.read()

    return HttpResponse(content, content_type="text/plain; charset=utf-8")
```

Стъпка 6) Реализирайте защитената версия за четене на файл

Добавете защитена версия на същата функционалност, при която пътят до файла се изгражда чрез os.path.join, преобразува се до абсолютен път и след това се проверява дали остава в рамките на директорията uploads. По този начин се предотвратява достъпът до файлове извън разрешената област, дори когато потребителят подава стойности като ../.

```
def read_file_safe(request):
    filename = request.GET.get("file")

    if not filename:
        return HttpResponse("No file specified.")

    base_dir = os.path.abspath("uploads")
    file_path = os.path.abspath(os.path.join(base_dir, filename))

    if not file_path.startswith(base_dir):
```

```

        return HttpResponse("Access denied")

    if not os.path.exists(file_path):
        return HttpResponse("File not found")

    with open(file_path, "r", encoding="utf-8") as f:
        content = f.read()

    return HttpResponse(content, content_type="text/plain; charset=utf-8")

```

Стъпка 7) Конфигурирайте URL адресите на приложението

Добавете URL маршрути за началната страница, за двете помощни страници с HTML форми, както и за двата изгледа, които реално извършват четенето на файлове. Така приложението ще има ясна структура и ще позволява лесно преминаване между уязвимата и защитената реализация.

Създайте файл `file_app/urls.py`:

```

from django.urls import path

from file_app.views import read_file_vulnerable, read_file_safe, home,
vulnerable_page, safe_page

urlpatterns = [
    path('/', home, name='home'),
    path('vulnerable/', vulnerable_page, name='vulnerable_page'),
    path('safe/', safe_page, name='safe_page'),
    path('read-file-vulnerable/', read_file_vulnerable,
name='read_file_vulnerable'),
    path('read-file-safe/', read_file_safe, name='read_file_safe'),
]

```

Стъпка 8) Свържете URL адресите на приложението с основния проект

Добавете маршрутизацията на приложението към основната URL конфигурация на проекта, така че всички адреси, дефинирани във `file_app`, да бъдат достъпни от браузъра. Това е стандартна стъпка при организиране на Django приложения с отделен `urls.py` файл.

```

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('file_app.urls')),
]

```

Стъпка 9) Стартирайте приложението и тествайте нормалната работа

Стартирайте сървъра и първо тествайте дали приложението работи коректно при подаване на валидно име на файл. Това позволява да се потвърди, че и двете версии изпълняват основната си функционалност преди да се премине към опити за експлоатиране на уязвимостта.

Стъпка 10) Демонстрирайте Path Traversal атака срещу уязвимата версия

Подайте стойност, която съдържа изкачване нагоре в дървото на директориите, например ../manage.py, и наблюдавайте как уязвимата версия връща съдържание на файл извън директорията uploads. Това показва, че приложението се доверява на потребителския вход и не ограничава достъпа до позволената област на файловата система.

Стъпка 11) Тествайте същата атака срещу защитената версия

Подайте същата стойност към защитената версия и сравнете резултата. Очакваното поведение е заявката да бъде блокирана с отговор Access denied, тъй като пътят след нормализация вече няма да започва с базовата разрешена директория uploads.

Стъпка 12) Анализирайте разликата между двете реализации

Сравнете поведението на уязвимата и защитената версия и анализирайте как липсата на контрол върху файловите пътища води до неоторизиран достъп до ресурси на сървъра. Обърнете внимание, че сигурната версия не разчита само на забрана на определени символи, а валидира крайния абсолютен път и проверява дали той принадлежи на разрешената директория.

III. Извод

В рамките на упражнението се демонстрира как неправилната обработка на файлови пътища позволява реализиране на Path Traversal атака и достъп до файлове извън предназначенията директория. Показано е, че директното използване на входни данни от потребителя при работа с файловата система създава сериозен риск за сигурността, докато проверката на абсолютния път спрямо разрешена базова директория представлява ефективен начин за ограничаване на достъпа.

